

Types of Research Software Tests

October 6, 2026

Contents

About this document	1
How to read a type	2
Terminology	2
At a glance	3
1 Scope: what runs	3
1.1 Unit test	3
1.2 Integration test	4
1.3 End-to-end test	4
2 Inputs: what goes in	4
2.1 Hand-picked examples	4
2.2 Edge cases	5
2.3 Generated inputs	5
2.4 Simulated data	6
2.5 Real data	6
3 Oracle: what the result is checked against	6
3.1 It runs	7
3.2 Snapshot	7
3.3 Reproducibility	7
3.4 Known answer	8
3.5 Invariant	8
3.6 Metamorphic relation	8
3.7 Round trip	9
3.8 Second opinion	9
3.9 Parameter recovery	9
3.10 Ground truth	10
3.11 Order of convergence	10
4 Tests by name	11

About this document

This document describes the types of tests for research software: what each type is, what it catches, and what it misses. It belongs to the same set as *Design Principles for Research Software*, cited below as *Design Principles*, *Testing Research Software*, and *Documenting Research Software*, and it is the one place where the set defines the terms of testing. The other documents use them with the meaning given here.

Every test makes three choices: its scope, what runs (Section 1); its inputs, what goes in (Section 2); and its oracle, what the result is checked against (Section 3). Each subsection of these three sections

describes one type, and a test takes one type from each section. A property-based test, for example, is usually a unit test on generated inputs with an invariant as its oracle. The three choices are largely independent: a unit test can use any oracle, and most oracles can check a single function as well as a whole workflow. Some combinations do not work, though, such as generated inputs with a known answer, because an arbitrary input has none; the field “Pairs with” of each type says which.

A test checks one claim, and each oracle sees some errors and is blind to others. One test, however well chosen, therefore checks only one aspect of an entry point, and an entry point is tested by several tests with different oracles. How many, and which, is the subject of *Testing Research Software*, “A test is only as good as its oracle”.

Many tests are known by a name that is not a type of its own, such as smoke test or regression test. Such a name stands for a combination of types, or for the purpose a test serves; Section 4 lists common ones.

This document states no rules. Where a type comes with a requirement or a default, such as the tolerance of a comparison of floating-point numbers, its field “See” cites the principle that states it.

How to read a type

Every type is described by the same fields, in the same order:

Definition What the type is.

Catches The errors it finds.

Misses The errors it cannot find, and so the reason to combine it with other types.

Cost The effort to write it, the time to run it, and the work to keep it up to date.

Pairs with The types of the other two sections that it works with, and those it does not.

Example One test of the type, from a scientific field.

See The principles that state rules about the type, and related types.

Terminology

The terms defined in *Design Principles*, such as package, library code, workflow code, entry point, host stack, and caller, carry the same meaning here. The terms of testing are defined here, for all documents of the set:

Test Code that runs part of the package on an input and checks the result automatically, with the outcome pass or fail.

Test suite All tests of a package, run together by the test runner of the host stack, such as `testthat` in R or `pytest` in Python.

Oracle The source of the expected value that a test compares a result with, or of the property that it checks (Section 3).

Tolerance The largest difference between a result and its expected value that a test accepts.

Each type, such as unit test, edge case, or snapshot, is defined in its own subsection of Sections 1 to 3, and the common names of tests, such as smoke test, in Section 4.

At a glance

Every type of this document in one line. The number and the title link to the type. A test takes one type from each of the three sections.

1 Scope: what runs

- | | | |
|-----|----------------------------------|--|
| 1.1 | Unit test | One function, or a few that work closely together, called directly. |
| 1.2 | Integration test | Several parts run together, to check that they fit. |
| 1.3 | End-to-end test | An entry point run the way its caller runs it, from start to finish. |

2 Inputs: what goes in

- | | | |
|-----|--------------------------------------|--|
| 2.1 | Hand-picked examples | Small, typical inputs that the author of the test writes into it. |
| 2.2 | Edge cases | Empty, single, tied, missing, extreme, and nearly singular inputs. |
| 2.3 | Generated inputs | Many inputs, produced by a program from the documented input domain. |
| 2.4 | Simulated data | Data drawn from the model, with parameters known exactly. |
| 2.5 | Real data | Data from a real measurement or collection, or an excerpt of it. |

3 Oracle: what the result is checked against

- | | | |
|------|--------------------------------------|---|
| 3.1 | It runs | The code finishes without an error; the result is not checked. |
| 3.2 | Snapshot | The result agrees with a stored output of an earlier run. |
| 3.3 | Reproducibility | The result agrees with that of another run of the same code. |
| 3.4 | Known answer | The result agrees with an exact value obtained without the code. |
| 3.5 | Invariant | The result has a property that every correct result has. |
| 3.6 | Metamorphic relation | A known change of the input changes the result in a known way. |
| 3.7 | Round trip | An operation followed by its inverse returns the input. |
| 3.8 | Second opinion | The result agrees with that of an independent implementation. |
| 3.9 | Parameter recovery | The method recovers the parameters behind simulated data, with honest uncertainty. |
| 3.10 | Ground truth | The result agrees with a truth about real data that was established without the method. |
| 3.11 | Order of convergence | The error falls with the resolution at the rate of the method. |
-

1 Scope: what runs

The scope of a test is the part of the package that it runs. The smaller the scope, the faster the test and the more precisely a failure points to its cause; the larger the scope, the more of the connections between the parts the test checks.

1.1 Unit test

- | | |
|-------------------|--|
| Definition | A test that calls one function, or a few functions that work closely together, directly and apart from the rest of the package. |
| Catches | Errors in the logic of that function, such as a wrong formula, sign, index, or convention. A failing unit test points to the function that broke. |
| Misses | Errors in how the parts fit together, such as a result passed on in the wrong units or order, a file that a later step cannot read, or an option that never reaches the function. |
| Cost | The lowest of the three scopes: on a tiny input, a unit test finishes in a fraction of a second, so hundreds of them run in minutes. It needs code split into functions that can be called on their own. |
| Pairs with | Every type of input and of oracle. Hand-picked examples (2.1) with known answers (3.4) make the classic unit test; generated inputs (2.3) with invariants (3.5) make a property-based test. |
| Example | The log-likelihood of three observations under a normal distribution, compared with the value worked out by hand. |

See *Testing Research Software*, “Many small tests, few big ones”.

1.2 Integration test

Definition	A test that runs several parts together, to check that they fit: the result of one function passed to the next, a file written by one step and read by another, or a call into an external library, such as a solver or a reader of a file format. Its scope lies between that of a unit test and that of an end-to-end test.
Catches	Mismatches at the interfaces, such as units or axis orders that differ between the parts, identifiers lost on the way, and assumptions about a dependency that do not hold, such as the meaning of its tolerance argument.
Misses	Errors inside a part that the combination happens to hide, and errors in the configuration and the glue of a whole entry point.
Cost	Higher than for a unit test: it runs longer, it needs inputs that suit every part, and a failure points to several parts at once.
Pairs with	Hand-picked examples (2.1) and simulated data (2.4); invariants (3.5), such as counts and identifiers that survive each interface; round trips (3.7) through a file format.
Example	The peaks found by a peak detector, passed to the function that fits them, keep the identifiers and the units of their detection.
See	<i>Design Principles</i> , “Derive abstractions from applications, not applications from abstractions”, on keeping the driving applications as integration tests.

1.3 End-to-end test

Definition	A test that runs an entry point the way its caller runs it, from start to finish: a workflow or a command, from its input files and configuration to its outputs, or, for library code, the calls of a complete analysis, such as a documented example.
Catches	Steps that do not fit together, files that a step cannot find or read, options of the configuration that never reach their step, and errors in code that no unit test runs.
Misses	Where an error is: a failure says that something broke, not what. Also every input and branch that the small dataset does not reach.
Cost	Minutes rather than seconds, and a small dataset that reaches every step and every branch of the configuration.
Pairs with	Simulated data (2.4) with parameter recovery (3.9) as its oracle; invariants (3.5), such as the number of samples and the total of the counts; snapshots (3.2) of the outputs. With “it runs” (3.1) as its only oracle, it is a smoke test (Section 4).
Example	A workflow run on six simulated samples: every sample appears in the final table, and the estimated fold changes recover the simulated ones.
See	<i>Design Principles</i> , “Test the whole workflow on a small dataset”; <i>Testing Research Software</i> , “Many small tests, few big ones”.

2 Inputs: what goes in

The inputs decide which part of the input space a test explores. Inputs that a person chooses are few and go where the person looks, which is rarely where the bugs are. Inputs that a program generates reach much more of the space, but an arbitrary input comes without a known answer, so the inputs also decide which oracles a test can use.

2.1 Hand-picked examples

Definition	Small inputs that the author of a test chooses and writes into it, usually typical ones: a few observations, a short sequence, a three-by-three matrix.
-------------------	---

Catches	Errors on typical inputs, which are the inputs that most users bring. A small input makes the claim of the test readable and its expected value easy to derive.
Misses	Everything the author did not think of. People choose inputs that they expect to work, so hand-picked examples rarely reach the inputs where code breaks.
Cost	Low to write and to run.
Pairs with	Known answers (3.4) worked out by hand; every scope, and most other oracles.
Example	A two-by-two table of counts, small enough that the p-value of Fisher’s exact test can be computed by hand.
See	Edge cases (2.2) and generated inputs (2.3), which reach what hand-picked examples miss.

2.2 Edge cases

Definition	Inputs at the edges of the input space: in size, such as an empty input and a single element; in value, such as zeros, ties, and the boundaries of a parameter’s range; missing and special values, such as NA, NaN, and infinities; extreme magnitudes and nearly singular inputs; and unusual structure, such as unsorted input.
Catches	Crashes, silent NaNs, off-by-one errors, divisions by zero, and wrong handling of missing values: code breaks at the edges, and typical data never go there. Writing the test also reveals edges where the behavior is not documented.
Misses	Errors in the interior of the input space, on typical data.
Cost	Low: the edges of most functions follow from a short list, and each test is fast.
Pairs with	Known answers (3.4), including the documented error or warning as the expected outcome; invariants (3.5). Generated inputs (2.3) search the edges automatically.
Example	The correlation of a variable with zero variance, which is undefined: the result is the documented one, such as NA, or the documented error.
See	<i>Testing Research Software</i> , “Most bugs live at the edges”; <i>Design Principles</i> , “Raise, warn, or stay silent”; <i>Design Principles</i> , “Mind the gap, don’t fill it”.

2.3 Generated inputs

Definition	Many inputs, produced by a program from a description of the input domain: random draws, systematic grids, or the inputs of a property-based testing framework, which also searches the edges and reduces a failing input to a minimal one.
Catches	Failures on inputs that no one thought of, in rare regions of the input domain and in unusual combinations of values.
Misses	Errors that the oracle cannot see, because an arbitrary input has no known answer, and inputs outside the domain that the generator describes.
Cost	Moderate: describing the input domain and finding an oracle that holds for every input take thought, and a test runs the code hundreds of times.
Pairs with	Invariants (3.5), metamorphic relations (3.6), round trips (3.7), and second opinions (3.8), which need no known answer; “it runs” (3.1), in a fuzz test. Known answers (3.4) only where a formula gives the answer for every input.
Example	Data frames of random size and content, generated by a property-based testing framework, on which reordering the rows leaves an estimate unchanged.
See	<i>Testing Research Software</i> , “Let the computer pick the inputs”; <i>Testing Research Software</i> , “Flaky tests cry wolf”, on randomness in tests.

2.4 Simulated data

Definition	Data drawn from the model that the method assumes, with parameters chosen by the test, so that the truth behind the data is known exactly. Unlike generated inputs, simulated data look like the data of an application, in size and in structure.
Catches	Errors of the whole method on data of realistic size and structure, such as bias and standard errors that are too small.
Misses	Failures on data that depart from the model, such as outliers, artifacts, and the quirks of real files, and every error that the simulator shares with the method, such as the same wrong parameterization.
Cost	Moderate to high: the simulator is written from the definition of the model, apart from the code of the method, and studies over many simulated datasets are slow.
Pairs with	Parameter recovery (3.9), which needs it; invariants (3.5); end-to-end tests (1.3) of workflows, whose small dataset it can provide with a known result.
Example	Counts for a few hundred genes, drawn from a negative binomial distribution with a known mean and dispersion for each gene.
See	<i>Testing Research Software</i> , “Plant the answer, then find it”.

2.5 Real data

Definition	Data from a real measurement, experiment, or collection, or an excerpt of it small enough for the repository.
Catches	What simulated data leave out: quirks of file formats, codes for missing values, unexpected identifiers, outliers, artifacts of instruments, and departures from the assumptions of the model.
Misses	Whether the result is right, unless the data come with a known answer or a ground truth. Without one, a test on real data shows only that the code runs, or that its result has not changed. Where there is a truth, it is often uncertain and covers only part of the data.
Cost	Finding data that are small, licensed for redistribution, and not confidential. Larger datasets make slow tests.
Pairs with	Ground truth (3.10), where the data have one; known answers (3.4), where a result of the computation is certified, as for the Longley data among the Statistical Reference Datasets of NIST; otherwise “it runs” (3.1), snapshots (3.2), invariants (3.5), and metamorphic relations (3.6). With a known signal planted into them, real data serve parameter recovery (3.9) as well.
Example	A few hundred lines of a public file of weather station records, with its codes for missing values and its irregular timestamps, to test the reader of the format.
See	<i>Testing Research Software</i> , “Every test stands on its own”, on licenses and confidential data.

3 Oracle: what the result is checked against

The oracle decides what a passing test shows. The first three oracles check only that the code runs, or compare the code with itself: they show that a result exists or has not changed, not that it is right. The others check the result against something independent of the code under test, and each sees a different kind of error. *Testing Research Software*, “A test is only as good as its oracle” lists strong oracles for common kinds of code.

3.1 It runs

Definition	The test passes if the code finishes without an error. Its result is not checked, or only its form, such as the number of output files or the shape of an array.
Catches	Crashes, missing dependencies, broken installations, and steps that do not fit together because a file is missing or has the wrong format.
Misses	Every wrong result that does not crash, which is the typical failure of research software: the plausible, wrong number.
Cost	The lowest of all oracles.
Pairs with	End-to-end tests (1.3) on a small hand-picked example, as a smoke test, and generated inputs (2.3), as a fuzz test (Section 4). It adds little to a unit test, whose result is cheap to check.
Example	A workflow run on a small dataset finishes and writes every output file it promises.
See	<i>Design Principles</i> , “Test the whole workflow on a small dataset”, which asks for more than this oracle: it requires checking the outputs, not only that the run finished. <i>Documenting Research Software</i> , “Every example runs”, which runs the examples of the documentation with this oracle or a snapshot.

3.2 Snapshot

Definition	The result is compared with a reference output stored from an earlier run of the same code. A test with this oracle is a snapshot test.
Catches	Silent changes, such as numbers that move after a refactoring, an update of a dependency, or a new compiler. For a large output, such as a table, a file, or a figure, it is often the only practical test.
Misses	Whether the result is right. A snapshot shows only that the result has not changed, and a snapshot of a wrong result protects the error.
Cost	Low to create. Every intended change of the result needs a decision and an update of the reference output, and a comparison bit for bit breaks on every new platform.
Pairs with	Every scope; fixed inputs, such as hand-picked examples (2.1) and real data (2.5). It shows more when the reference output was checked against another oracle before it was stored.
Example	The table of model estimates for a reference dataset, compared within a tolerance with the table stored at the last release.
See	<i>Testing Research Software</i> , “Unchanged is not correct”; <i>Testing Research Software</i> , “Compare floats with tolerance”; <i>Documenting Research Software</i> , “Every example runs”, for the output shown in the documentation.

3.3 Reproducibility

Definition	The result is compared with that of another run of the same code on the same input: a run with the same seed, with another number of threads or processes, on another platform, or with other versions of the dependencies.
Catches	Hidden nondeterminism, such as an unset seed, a race condition, uninitialized memory, or a dependence on global state or on the order in which files are listed; and results that change across platforms or versions of the dependencies by more than the tolerance.
Misses	Every error that is the same in each run: a wrong result, reproduced, is still wrong.
Cost	Low: the code runs twice. A comparison across platforms needs automatic test runs on each of them.
Pairs with	Every scope and every type of input; it matters most for stochastic and parallel code.

Example	A Monte Carlo simulation run twice with the same seed, then on one and on eight threads, with the results compared within the tolerance that defines reproducibility.
See	<i>Design Principles</i> , “Once is luck, twice is science”; <i>Testing Research Software</i> , “Flaky tests cry wolf”.

3.4 Known answer

Definition	An exact result for a particular input, obtained without the code under test: an analytic solution or a limiting case, a tiny case worked out by hand or with computer algebra, a certified reference value, or a published value.
Catches	Wrong formulas, constants, signs, and conventions, on the inputs where the answer is known.
Misses	Everything away from those inputs. Known answers exist mostly for small or simple inputs, the easy corner of the input space. A reference that uses another convention, such as a variance divided by n rather than $n - 1$, raises false alarms.
Cost	Low when the answer is in a table, higher when it has to be derived.
Pairs with	Hand-picked examples (2.1) and edge cases (2.2); real data (2.5) with certified results; every scope. Generated inputs (2.3) only where a formula gives the answer for every input.
Example	The sample variance of 1, 2, 3, and 4, which is $5/3$ with the divisor $n - 1$ and $5/4$ with the divisor n .
See	<i>Testing Research Software</i> , “Know the answer before you ask”; <i>Design Principles</i> , “No magic numbers”.

3.5 Invariant

Definition	A property that every correct result has, whatever the input, such as a conservation law, a normalization, a bound, a symmetry, or a consistency relation.
Catches	Errors that break the structure of a result: probabilities that do not sum to one, a negative variance, mass that is lost, a distance matrix that is not symmetric. It needs no known answer, so it can be checked on any input.
Misses	A wrong result that still has the property: probabilities that sum to one can still be the wrong probabilities.
Cost	Low once the property is known, and fast to check.
Pairs with	Generated inputs (2.3), as a property-based test; simulated (2.4) and real data (2.5); end-to-end tests (1.3), as counts and totals that every step preserves.
Example	In a simulation of a closed system, the total energy stays constant, within the error of the integrator, over every time step.
See	<i>Testing Research Software</i> , “Test invariants, not only examples”; <i>Design Principles</i> , “Wear belt and braces”, on checking the same invariants on every run.

3.6 Metamorphic relation

Definition	A relation between the results for two related inputs: the input is changed in a way whose effect the method determines, and the result changes accordingly. Examples are reordering, rescaling, duplicating, and splitting the data, and monotonicity in a parameter.
Catches	Errors that leave each single result plausible, such as swapped axes, identifiers misaligned after a sort, a unit converted twice, or a result that depends on the order of the rows.
Misses	Errors that respect the relation: a constant bias survives every reordering.

Cost	Moderate: finding relations takes knowledge of the method, and the code runs twice. A relation that holds only approximately, for example because reordering changes the rounding of a sum, needs a tolerance of its own.
Pairs with	Generated inputs (2.3), simulated data (2.4), and real data (2.5), because it needs no known answer; every scope.
Example	The coordinates of a set of points, converted from meters to kilometers: their estimated spread falls by a factor of 1000, and permuting the points leaves it unchanged.
See	<i>Testing Research Software</i> , “Change the input, predict the output”.

3.7 Round trip

Definition	An operation followed by its inverse returns the original input: a file written and read back, a transform and its inverse, a unit converted and converted back, a value encoded and decoded. It is a special metamorphic relation (3.6).
Catches	Information lost on the way, such as precision, units, identifiers, or missing values; wrong encodings; and axes swapped in one of the two directions.
Misses	Errors that the two directions share and that cancel: a transform with the wrong normalization returns its input perfectly if its inverse has the matching error.
Cost	Low where both directions exist, and fast.
Pairs with	Generated inputs (2.3) and edge cases (2.2), such as empty tables and missing values; integration tests (1.2) across a file format; real data (2.5) in that format.
Example	A table with missing values, units, and sample identifiers, written to the package’s file format, read back, and compared with the original.
See	<i>Design Principles</i> , “Nothing lost in translation”.

3.8 Second opinion

Definition	The result is compared with that of an independent implementation of the same method: a naive one written for the test, such as a plain loop instead of vectorized code, a dense matrix instead of a sparse one, or a direct sum instead of a fast transform; or an established package, possibly in another language.
Catches	Errors of optimized code, such as wrong indices in vectorized or sparse code and approximations of fast algorithms that go wrong, on every input where the two implementations disagree.
Misses	Errors that both implementations share, because they come from the same derivation, the same paper, or the same person. A disagreement shows that one of the two is wrong, not which.
Cost	A naive implementation is quick to write but slow, so it works on small inputs only. An established package adds a test dependency, and a new release of it can change its numbers.
Pairs with	Generated inputs (2.3), because it provides an expected value for every input; hand-picked examples (2.1); unit tests (1.1).
Example	The product of two small random sparse matrices, compared with the product of the same matrices stored as dense ones.
See	<i>Testing Research Software</i> , “Get a second opinion”.

3.9 Parameter recovery

Definition	Data are simulated from the model with known parameters, the method is applied to them, and the test checks that it recovers the parameters with honest uncertainty: over
-------------------	---

	many simulated datasets, the estimates scatter as much as the reported standard errors say, and the intervals contain the true values at their nominal rate, their coverage.
Catches	Bias, standard errors and intervals that are too narrow, tests that reject too often under the null hypothesis, and failures in hard cases, such as small samples, parameters at the boundary of their range, and weak signals.
Misses	Errors that the simulator shares with the method, and failures on data that depart from the model.
Cost	High: an independent simulator, bounds derived from the sampling error, and many replicates, which make studies of coverage slow.
Pairs with	Simulated data (2.4), which it needs, or real data (2.5) with a planted signal; integration (1.2) and end-to-end tests (1.3).
Example	A thousand datasets simulated from a mixed model with known variance components: the 95% intervals contain the true values in about 95% of them, within the sampling error of a thousand replicates.
See	<i>Testing Research Software</i> , “Plant the answer, then find it”; <i>Testing Research Software</i> , “Flaky tests cry wolf”, on bounds for statistical claims.

3.10 Ground truth

Definition	The result is compared with a truth about real data that was established without the method: by the design of an experiment, by an independent measurement, or by expert annotation. Examples are truth sets for variant calling, spike-in controls of known concentration, mock communities of known composition, imaging phantoms, and annotated images. The comparison is usually a summary, such as a sensitivity or an error rate within a bound. A certified result of a computation is a known answer (3.4) instead: it says what the computation gives, not what is true.
Catches	Failures that only real data show, such as those caused by noise, artifacts, and departures from the model, and changes in performance after a change of the code.
Misses	Errors smaller than the uncertainty of the truth, and errors where the truth is unknown; truth sets often cover only the regions where it is certain. A failure may be a limit of the method rather than a bug, since a correct implementation can miss the truth: the test checks the implementation when its bound comes from the known performance of the method, while whether the method suits the data is validation.
Cost	High: data with a known truth are rare, expensive to make, and often large, which makes slow tests, and their license has to allow redistribution.
Pairs with	Real data (2.5), which it needs; end-to-end tests (1.3).
Example	A variant caller run on a region of a reference genome with a published truth set, with its sensitivity and precision compared with the values published for the method.
See	<i>Testing Research Software</i> , “About this guide”, on testing and validation; <i>Testing Research Software</i> , “Know the answer before you ask”, on tolerances derived from the precision of a reference.

3.11 Order of convergence

Definition	A discretization is run at a sequence of resolutions against an exact solution, and the rate at which its error falls is compared with the order of the method. Where no exact solution exists, the method of manufactured solutions provides one. The same idea applies to Monte Carlo estimates, whose error falls as $N^{-1/2}$ in the number of samples, and to iterative solvers.
-------------------	--

Catches	Bugs that lower the order of the method, such as a boundary condition applied one step late or a wrong coefficient, which still give a small error on a fine enough grid and so pass a test at one resolution.
Misses	Errors in terms that the exact solution does not exercise, such as a term with a derivative that vanishes for the chosen solution, and errors of the model itself: the right equations, solved correctly, can still be the wrong physics.
Cost	High: several runs, some at fine resolution, at resolutions fine enough for the leading term of the error to dominate and coarse enough for rounding error not to.
Pairs with	Unit (1.1) and integration tests (1.2) of solvers, quadrature rules, and finite-difference schemes, on a problem with an exact or a manufactured solution.
Example	Halving the step of a second-order solver for an ordinary differential equation with a known solution divides the error by about four.
See	<i>Testing Research Software</i> , “Check the order of convergence”.

4 Tests by name

Many tests are known by a name that is not a type of its own. Some names stand for a combination of types, such as property-based test; others add the purpose a test serves or the moment it runs, such as regression test and smoke test. Table 1 lists common ones, with the types that they usually combine and what the name adds. The numbers refer to the subsections of this document.

Some names belong to activities that are not tests of results. Mutation testing and code coverage check the test suite itself (*Testing Research Software*, “A test that cannot fail tests nothing”), benchmarks measure performance (*Design Principles*, “Size matters”), and validation asks whether the method fits the scientific question, which no test can answer (*Testing Research Software*, “About this guide”).

Table 1: Common names of tests, with the types that they usually combine.

Name	Scope	Inputs	Oracle	What the name adds
Benchmark problem	End-to-end (1.3)	A standard problem of the field (2.1)	Known answer (3.4), or the results of other codes (3.8)	A problem that a community agreed on, such as a shock tube in fluid dynamics, so that codes can be compared. Not a benchmark of performance.
Doc test	Unit (1.1)	The example in the documentation (2.1)	Its printed output, as a snapshot (3.2), or it runs (3.1)	Keeps the examples of the documentation working: doctest in Python compares their printed output, and R CMD check runs the examples of an R package.
Fuzz test	Unit or end-to-end	Generated (2.3), often malformed	It runs (3.1), or the documented error	Searches for inputs that crash the code or slip past its validation; runs for hours, apart from the test suite.
Golden master, characterization test, approval test	End-to-end (1.3)	Hand-picked examples (2.1) or real data (2.5)	Snapshot (3.2)	Records the behavior of existing code, often before code without tests is changed. In an approval test, a person approves each new reference output.
Gradient check	Unit (1.1)	Hand-picked or generated points (2.3)	Second opinion (3.8): finite differences	Checks a derivative, coded by hand or computed by automatic differentiation, against finite differences of the function.

Name	Scope	Inputs	Oracle	What the name adds
Manufactured solution	Unit or integration (1.2)	A problem built from a chosen solution	Order of convergence (3.11)	Provides an exact solution where none is known: the chosen function, inserted into the equations, gives the source terms that it requires.
Property-based test	Unit (1.1)	Generated (2.3)	Invariant (3.5), metamorphic relation (3.6), or round trip (3.7)	A framework, such as Hypothesis for Python or quickcheck for R, generates the inputs, searches the edges, and reduces a failing input to a minimal one.
Regression test	Any	The input that exposed a bug	Any; often a known answer (3.4)	Added when a bug is fixed, so that the bug cannot return unnoticed. Some communities also call snapshot tests regression tests.
Simulation study	Integration or end-to-end	Simulated data (2.4), many datasets	Parameter recovery (3.9)	Measures bias and coverage over many simulated datasets; slow, so it runs apart from the fast tests. Its coverage is that of intervals, not of code.
Smoke test	End-to-end (1.3)	A small hand-picked example (2.1)	It runs (3.1)	Runs first, after a build, an installation, or a change of environment, to decide whether the other tests are worth running.